

Shallow Quantum Circuit Implementation of Symmetric Functions With Limited Ancillary Qubits

Wei Zi^{1b}, Junhong Nie^{1b}, and Xiaoming Sun^{1b}, *Senior Member, IEEE*

Abstract—Optimizing the depth and number of ancillary qubits in quantum circuits is crucial in quantum computation, given the limitations imposed by current quantum devices. In this article, we introduce an innovative approach for implementing arbitrary symmetric Boolean functions using poly-logarithmic depth quantum circuits with only a logarithmic number of ancillary qubits. Symmetric functions are those whose outputs are dictated solely by the Hamming weight of the inputs. These functions find applications across various domains, including quantum machine learning and arithmetic circuit synthesis. Moreover, by fully leveraging the potential of qutrits, the ancilla count can be further reduced to just one. The key technique involves a novel poly-logarithmic depth quantum circuit designed to compute Hamming weight without the need for ancillary qubits. This quantum circuit for Hamming weight is of independent interest due to its wide-ranging applications, such as in quantum memory, quantum machine learning, and Hamiltonian dynamics simulations.

Index Terms—Quantum circuit, quantum circuit optimization, quantum circuit synthesis, quantum computing, symmetric function, upper bound.

I. INTRODUCTION

QUANTUM computing has attracted considerable attention for its potential to surpass classical computing, exemplified by Shor's algorithm [1]. There have been notable advancements in hardware implementations and quantum computing software optimizations in recent years. One of the most critical research directions is the optimization of quantum circuits [2], [3], [4]. The optimization of circuit depth and ancilla count has garnered significant attention from researchers, as these two measures correspond to the time and space complexity of quantum computation, respectively [5], [6], [7], [8], [9], [10], [11], [12].

Symmetric functions [13] are Boolean functions whose outputs depend solely on the Hamming weight of their inputs. These functions have wide-ranging applications in reversible circuit synthesis [14], quantum adder [15], and quantum machine learning [16]. The quantum circuit

TABLE I
COMPARISON OF CIRCUIT DEPTH AND ANCILLA COUNT OF QUANTUM CIRCUITS IMPLEMENTING SYMMETRIC FUNCTIONS

	circuit depth	ancilla count
[19], [20]	$O(n^2)$	$O(n^2)$
[21], [22]	$O(n^2)$	$O(n)$
[8]	$O(n \log n)$	$O(n)$
[7]	$O(\log^2 n)$	$O(n \log^2 n)$
[5]	$O(n^2)$	0
our result	$O(\log^2 n)$	$\lceil \log(n+1) \rceil$

implementation of symmetric functions has garnered attention from researchers [17], [18]. It is worth noting that some results referenced in this article aim to optimize the circuit size. In contrast, this article focuses on optimizing the circuit depth and ancilla count. Perkowski et al. [19], [20] proposed a quantum circuit with $O(n^2)$ depth to implement symmetric functions, utilizing $O(n^2)$ ancillary qubits, where n is the number of input qubits. Maslov [21], [22] maintained the same circuit depth while reducing the ancilla count to $O(n)$. Chattopadhyay and Baksi [8] devised a circuit with $O(n \log n)$ depth to implement symmetric functions with $O(n)$ ancillary qubits. Takahashi and Tani [7] were able to reduce the circuit depth to $O(\log^2 n)$. However, their approach requires $O(\log n)$ clean ancillary qubits and $O(n \log^2 n)$ borrowed ancillary qubits. Maslov et al. [5] implemented symmetric functions without ancillary qubits but at the expense of $O(n^2)$ circuit depth.

In this article, we introduce a novel approach to implementing arbitrary symmetric functions using quantum circuits of depth $O(\log^2 n)$, requiring only $\lceil \log(n+1) \rceil$ clean ancillary qubits. We are the first to demonstrate that symmetric functions can be implemented with lower circuit depth and ancilla count simultaneously. We provide a comparison of these results in Table I. It is worth noting that any Boolean function can be transformed into a symmetric function by repeating the input [23]. In [24], a method is proposed to optimize the number of repetitions for symmetrization, greatly extending the potential applications of our result. Our key technique is an efficient quantum circuit designed to compute the Hamming weight.

The Hamming weight of a bit-string is the count of 1's in the string, and the Hamming distance represents the number of positions at which two bit-strings of the same length

Received 20 June 2024; revised 12 December 2024; accepted 1 January 2025. Date of publication 5 February 2025; date of current version 21 July 2025. This work was supported in part by the National Natural Science Foundation of China under Grant 62325210 and Grant 92465202. This article was recommended by Associate Editor D. Atienza. (*Corresponding author: Xiaoming Sun.*)

The authors are with the State Key Laboratory of Processors, Institute of Computing Technology, Chinese Academy of Sciences, Beijing 100190, China, and also with the School of Computer Science and Technology, University of Chinese Academy of Sciences, Beijing 100049, China (e-mail: sunxiaoming@ict.ac.cn).

Digital Object Identifier 10.1109/TCAD.2025.3539002

TABLE II
COMPARISON OF CIRCUIT DEPTH AND ANCILLA COUNT OF QUANTUM CIRCUITS COMPUTE THE HAMMING WEIGHT

	circuit depth	ancilla count
[29], [31]	$O(n \log^2 n)$	$O(1)$
[8], [33]	$O(n)$	$O(n)$
[7]	$O(\log^2 n)$	$O(n \log^2 n)$
our result	$O(\log^2 n)$	0
our result	$O(\log n)$	$O(n)$

differ. Quantum circuit for Hamming distance is broadly applicable in quantum memory [25], and quantum machine learning [26], [27], particularly in quantum nearest neighbor classification algorithms [28], [29], [30], [31], [32]. Indeed, any efficient quantum circuit implementation of Hamming weight implies an efficient implementation for Hamming distance simply by adding two layers of CNOT gates. Therefore, the quantum circuit implementation of Hamming weight is of independent interest. To implement a quantum circuit for computing the Hamming weight of an n -bit string, $\lceil \log(n+1) \rceil$ output qubits are necessary to store the results and are therefore not included in the ancilla count. In [29] and [31], the Hamming weight is computed using a quantum circuit with a depth of $O(n \log^2 n)$ and $O(1)$ ancillary qubits. In [8] and [33], although the circuit depth is reduced to $O(n)$, the ancilla count increases to $O(n)$. In [7], the circuit depth is further reduced to $O(\log^2 n)$ but using $O(n \log^2 n)$ borrowed ancillary qubits.

In this article, we propose a quantum circuit with a depth of $O(\log^2 n)$ to compute the Hamming weight without the need for ancillary qubits. It achieves the best circuit depth and ancilla count simultaneously compared to previous results. Table II presents a comparison of results regarding Hamming weight.

The implementation of symmetric functions consists of two steps. In the first step, we construct a quantum circuit of depth $O(\log^2 n)$ to compute the Hamming weight of the input and store the result in $\lceil \log(n+1) \rceil$ clean ancillary qubits. This turns a symmetric function with n -bit input to a Boolean function with $\lceil \log(n+1) \rceil$ -bit input. In the second step, we treat the $\lceil \log(n+1) \rceil$ clean ancillary qubits as input qubits and the original n -qubit input qubits as borrowed ancillary qubits. By designing a parallelized quantum circuit with a depth of $O(\log^2 n)$, we are able to compute the output of the symmetric function on the target qubit.

We can further reduce the ancilla count to just one by incorporating an additional energy level (qutrit). In recent years, the implementation of qutrit and even qudit systems in experiments has made significant progress [34], [35], [36], [37], [38], [39], [40], [41]. These higher energy levels can be harnessed to enhance information density [10], [42] or serve as ancillary energy levels [9], [43]. Gokhale et al. [9] utilized qutrits to implement a generalized n -Toffoli gate with a depth of $O(\log n)$ without ancillary qutrit. We extend their findings and implement arbitrary symmetric functions with a circuit depth of $O(\log^2 n)$ using only one ancillary qutrit in

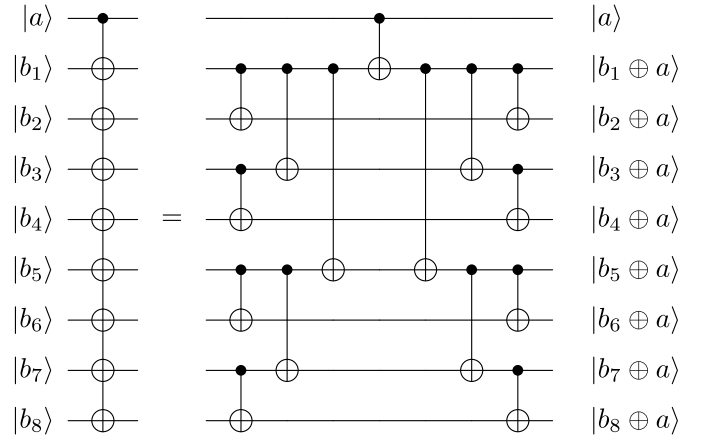


Fig. 1. Quantum circuit implementing the $\mathbb{F}_8$ gate.

qutrit systems. This expansion broadens the range of efficiently achievable functions in qutrit systems from the n -Toffoli gate to any symmetric function, albeit with a slight increase in circuit depth and the number of ancillary qubits. It is worth noting that by transforming the circuit in qutrit systems to qubit systems, we find a circuit with $O(\log n)$ depth and $O(n)$ ancillary qubits that can compute Hamming weight, this result is also presented in Table II. This result has direct applications in Hamiltonian dynamics simulation [44], [45], [46].

The subsequent sections of this article are structured as follows. Section II provides preliminaries. Sections III and IV detail the implementation of symmetric functions using quantum circuits in qubit and qutrit systems, respectively. Section V presents the analysis and evaluation of our results. Finally, Section VI concludes this article.

II. PRELIMINARY

We define the symmetric function and quantum circuit implementation of a Boolean function as follows.

Definition 1: A symmetric function $f : \{0, 1\}^n \rightarrow \{0, 1\}$ is a Boolean function that satisfies the following condition: for arbitrary permutation π of the n terms, we have $f(x_1, x_2, \dots, x_n) = f(x_{\pi(1)}, x_{\pi(2)}, \dots, x_{\pi(n)})$.

Note that the output of a symmetric function is determined solely by the Hamming weight of its input.

Definition 2: A quantum circuit C implementing a Boolean function $f : \{0, 1\}^n \rightarrow \{0, 1\}$ means that $C|x\rangle|a\rangle|t\rangle = |x\rangle|a\rangle|t \oplus f(x)\rangle$, where $x \in \{0, 1\}^n$, $|t\rangle$ and $|a\rangle$ represent the target qubit and ancillary qubits. Circuit C is composed of CNOT and single-qubit gates.

The $Z(\theta)$ gate is defined by: $Z(\theta) = \begin{pmatrix} 1 & 0 \\ 0 & e^{i\theta} \end{pmatrix}$.

The quantum fan-out gate $\mathbb{F}_n$ is defined by

$$\mathbb{F}_n|a\rangle|b_1\rangle|b_2\rangle \cdots |b_n\rangle = |a\rangle|b_1 \oplus a\rangle|b_2 \oplus a\rangle \cdots |b_n \oplus a\rangle.$$

The $Z(\theta)$ gate and quantum fan-out gate $\mathbb{F}_n$ will be used in Section III. There is a well-known result regarding the quantum circuit implementation of $\mathbb{F}_n$, which has been utilized in several papers [7], [47]. Therefore, we illustrate the circuit of $\mathbb{F}_8$ in Fig. 1 and omit the proof of Lemma 1.

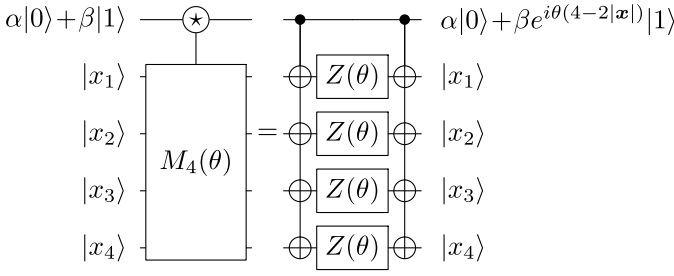


Fig. 2. Quantum circuit of the $C-M_4(\theta)$ gate.

Lemma 1: The quantum fan-out gate $\mathfrak{F}_n$ can be implemented using a quantum circuit of depth $O(\log n)$ without ancillary qubit.

This article introduces two types of ancillary qubits: clean and borrowed ancillary qubits. Clean ancillary qubits are promised to have a quantum state of $|0\rangle$ before use and it is required to recover them back to state $|0\rangle$ after use. Borrowed ancillary qubits can be in an arbitrary state before use and it is required that their quantum state remains unchanged after use.

A qutrit is a three-level quantum-mechanical system whose computation basis consists of states $|0\rangle$, $|1\rangle$, and $|2\rangle$. The definition of two types of ancillary qutrits is similar to that of qubits. When designing quantum circuits in qutrit systems, we use X_{ij} gate and $|0\rangle$ - X_{01} gate as elementary gates. The X_{ij} gate is defined as

$$X_{ij} = |i\rangle\langle j| + |j\rangle\langle i| + |k\rangle\langle k|$$

where $k \neq i$ and $k \neq j$. The $|0\rangle$ - X_{01} gate applies the X_{01} gate on the target qutrit if and only if the control qutrit state is $|0\rangle$. X_{+1} gate and X_{-1} gate will be utilized in Section IV. Specifically, $X_{+1} = X_{02}X_{01} = \sum_i |i+1 \bmod 3\rangle\langle i|$ and $X_{-1} = X_{+1}X_{+1}$.

III. EFFICIENT IMPLEMENTATION OF SYMMETRIC FUNCTIONS

We first introduce a quantum gate that can compute the Hamming weight of the input and store this value in the phase of the first qubit.

Definition 3: The function of quantum gate $C-M_n(\theta)$ is

$$(\alpha|0\rangle + \beta|1\rangle) \otimes |\mathbf{x}\rangle \rightarrow (\alpha|0\rangle + \beta e^{i\theta(n-2|\mathbf{x}|)} |1\rangle) \otimes |\mathbf{x}\rangle$$

where $\theta \in \mathbb{R}$, $|\mathbf{x}\rangle = |x_1, x_2, \dots, x_n\rangle$, $x_i \in \{0, 1\}$, $\alpha, \beta \in \mathbb{C}$, $|\alpha|^2 + |\beta|^2 = 1$, and $|\mathbf{x}| = \sum_i x_i$.

Lemma 2: For any $x_i \in \{0, 1\}$, $\theta \in \mathbb{R}$ and $\alpha|0\rangle + \beta|1\rangle$, the $C-M_n(\theta)$ gate can be implemented using a quantum circuit of depth $O(\log n)$ without ancillary qubit.

Proof: The quantum circuit of $C-M_n(\theta)$ can be decomposed into three steps. Both steps 1 and 3 are fan-out gates on $n+1$ qubits, while step 2 applies a $Z(\theta)$ gate to the last n qubits. An example of $C-M_4(\theta)$ is depicted in Fig. 2.

For the correctness of this circuit, consider the initial state as follows: $(\alpha|0\rangle + \beta|1\rangle) \otimes |x_1, \dots, x_n\rangle$. After step 1, the quantum state is as follows:

$$\alpha|0\rangle|x_1, \dots, x_n\rangle + \beta|1\rangle|x_1 \oplus 1, \dots, x_n \oplus 1\rangle.$$

After step 2, the quantum state is as follows (the global phase $e^{i\theta|\mathbf{x}|}$ is omitted):

$$\alpha|0\rangle|x_1, \dots, x_n\rangle + \beta e^{i\theta(n-2|\mathbf{x}|)} |1\rangle|x_1 \oplus 1, \dots, x_n \oplus 1\rangle.$$

After step 3, the quantum state is as follows:

$$(\alpha|0\rangle + \beta e^{i\theta(n-2|\mathbf{x}|)} |1\rangle) \otimes |x_1, \dots, x_n\rangle.$$

We can now ignore the global phase and conclude the proof of correctness. Note that the circuit depth is directly derived from Lemma 1. The global phase can also be eliminated by adding a fourth step to the circuit: applying one $Z(-\theta)$ gate on each of the last n qubits, which only increases the circuit depth by one. ■

A similar idea as Lemma 2 is utilized for implementing logic-AND in [44].

Note that the symbol of the $C-M_n(\theta)$ gate depicts one control qubit and n target qubits. However, in reality, only the relative phase of the “control qubit” is altered. This notation is used for convenience in representing quantum circuits. Next, we need to represent the information of $|\mathbf{x}|$, which is originally stored in the qubit phase, in the computational basis. Note that $\lceil \log(n+1) \rceil$ output qubits that initialized to $|0\rangle$ are necessary for storing all the bits of $|\mathbf{x}|$.

Theorem 1: The Hamming weight of an n -bit string can be computed by a quantum circuit C_n with depth $O(\log^2 n)$, without using ancillary qubits. More precisely

$$C_n|0\rangle^{\otimes \lceil \log(n+1) \rceil} |\mathbf{x}\rangle = ||\mathbf{x}|| |\mathbf{x}\rangle$$

where $\mathbf{x} \in \{0, 1\}^n$, $|\mathbf{x}| = \sum_i x_i$. The first $\lceil \log(n+1) \rceil$ qubits are output qubits.

Proof: The quantum circuit for C_n is shown in Fig. 3. The input $\mathbf{x} \in \{0, 1\}^n$, and let $m = \lceil \log(n+1) \rceil$. The binary representation of $|\mathbf{x}|$ can be expressed by $|\mathbf{x}|_{(2)} = b_1 b_2 \dots b_m$.

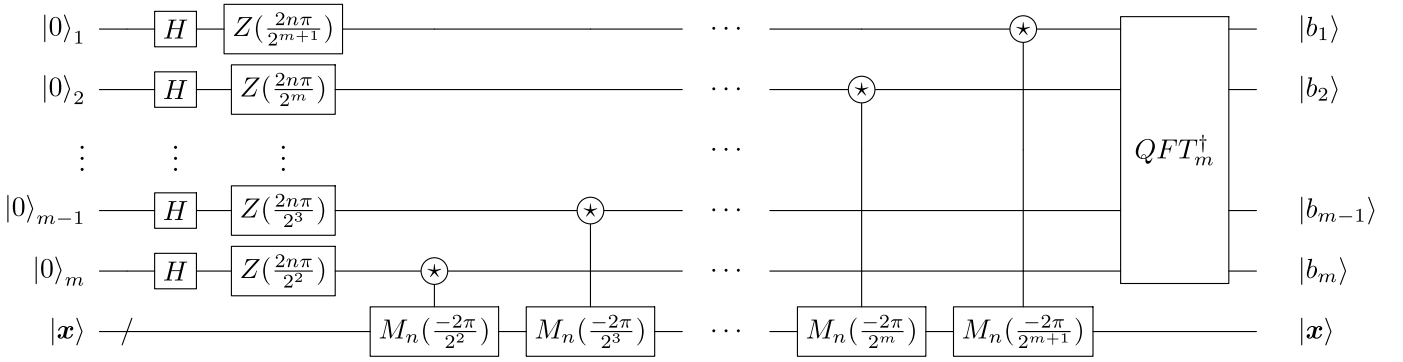
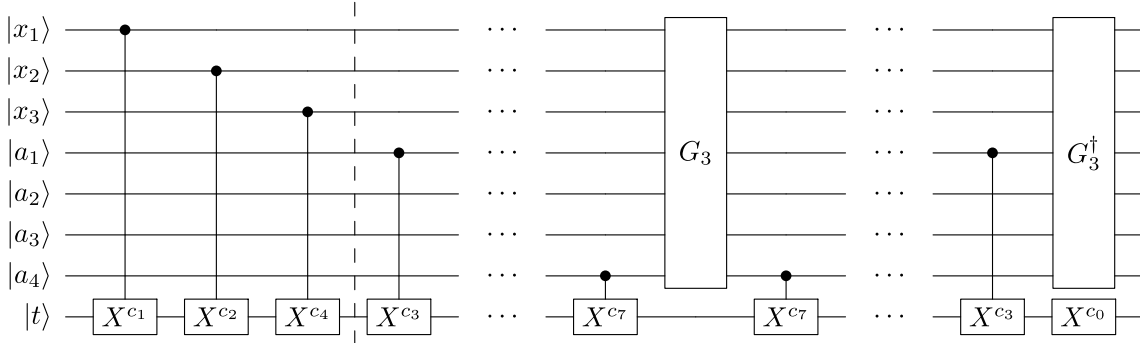
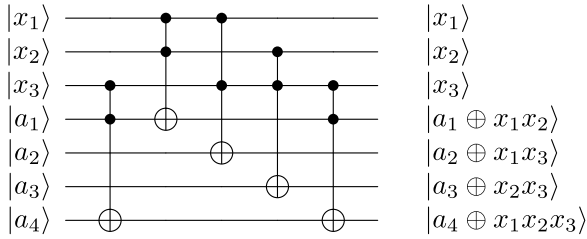
To verify its correctness, we consider a specific output qubit $|0\rangle_j$ for $j = 1, 2, \dots, m$. Initially, we apply an H gate and a $Z(2n\pi/2^{m-j+2})$ gate to this qubit. As a result, the quantum state is transformed to $|0\rangle \rightarrow (|0\rangle + e^{2in\pi/2^{m-j+2}} |1\rangle)/\sqrt{2}$.

Afterward, we apply a $C-M_n(-2\pi/2^{m-j+2})$ gate. According to Lemma 2, the quantum state is transformed to

$$\begin{aligned} & \left(|0\rangle + e^{\frac{2in\pi}{2^{m-j+2}} - \frac{2i\pi}{2^{m-j+2}} (n-2|\mathbf{x}|)} |1\rangle \right) / \sqrt{2} \\ &= \left(|0\rangle + e^{\frac{2ij|\mathbf{x}|\pi}{2^{m-j+1}}} |1\rangle \right) / \sqrt{2}. \end{aligned}$$

Note that we have already encoded the information of $|\mathbf{x}|$ into the phase of m output qubits. Next, we can use $QFT_m^\dagger$ to transpose $|\mathbf{x}|$ into computational basis. After applying $QFT_m^\dagger$ to the m qubits, the resulting quantum state is $|b_1\rangle|b_2\rangle \dots |b_m\rangle |\mathbf{x}\rangle = ||\mathbf{x}|| |\mathbf{x}\rangle$. Note that the depth of QFT_m is $O(m)$ [48]. Therefore, by Lemma 2, the depth of the circuit in Fig. 3 is $O(\log^2 n)$. ■

Our method can be viewed as utilizing Fourier-based arithmetic first introduced in [49]. The difference lies only in the computational goal. While [49] aims to compute the sum of two numbers, our focus in this article is on calculating the Hamming weight. Our method for computing the Hamming weight can also be considered as grounded in the phase


 Fig. 3. Quantum circuit of C_n . The input $x \in \{0, 1\}^n$, $m = \lceil \log(n+1) \rceil$. The binary representation of $|x|$ is $b_1 b_2 \dots b_m$.

 Fig. 4. Circuit that implements a Boolean function $f : \{0, 1\}^3 \rightarrow \{0, 1\}$.

 Fig. 5. Quantum circuit of G_3 .

estimation algorithm (PEA) [48]. The utilization of PEA for computing the Hamming weight is well known, as in previous research like [50]. However, we are the first to offer a low-depth circuit implementation of PEA for computing the Hamming weight without ancillary qubits.

Theorem 1 enables us to obtain the Hamming weight $|x|$ from the input x . For the majority function, we can easily implement by testing whether $|x| \geq n/2$. For general symmetric functions, we need a new method to implement Boolean functions using borrowed ancillary qubits.

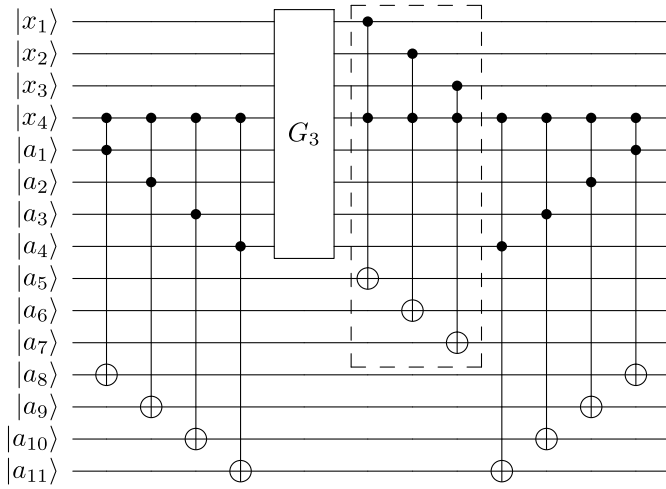
Lemma 3: A Boolean function $f : \{0, 1\}^n \rightarrow \{0, 1\}$ can be implemented using a quantum circuit of depth $O(n^2)$ with $2^n - n - 1$ borrowed ancillary qubits.

Proof: Assuming we can represent a Boolean function f using its algebraic normal form (ANF): $f(x) = \bigoplus_{j=0}^{2^n-1} c_j f_j(x)$, $c_j \in \{0, 1\}$, $f_j(x) = \bigwedge_{k=1}^n x_k^{b_k}$, $(j)_2 = b_n b_{n-1} \dots b_1$. There are also existing works that utilize ANF for resource-efficient quantum circuit construction of Boolean functions [51].

Note that the ANF form is an XOR of multiple terms, and there are at most 2^n different terms. Our approach is to XOR each term of the ANF form onto the target qubit $|t\rangle$, causing the target qubit $|t\rangle$ to become $|t \oplus f(x)\rangle$.

The quantum circuit that implements f is divided into 3 parts. The first part XORs all the linear terms onto the target qubit $|t\rangle$. For each $i \in \{1, 2, \dots, n\}$, if $c_{2^i} = 1$, we add a CNOT gate to the circuit with x_i as control qubit and $|t\rangle$ as target qubit, indicating that x_i is a term in the ANF form of f . The second part requires $2^n - n - 1$ borrowed ancillary qubits, where each ancillary qubit corresponds to a higher-order term. We add the gate G_n to the circuit to XOR each higher-order term with its corresponding ancillary qubit. If $c_j = 1$ for a higher-order term $f_j(x)$, we add a CNOT gate on both sides of G_n . The control qubit is the corresponding ancillary qubit for $f_j(x)$, and the target qubit is $|t\rangle$. Finally, we add $G_n^\dagger$ to restore all ancillary qubits. The third part adds an X gate applied on $|t\rangle$ to the end of the circuit if $c_0 = 1$. This ensures correctness by noticing that all terms of the ANF form have been XORed onto $|t\rangle$. Fig. 4 shows an example circuit for $n = 3$. The Left of the dashed barrier corresponds to the first part. The circuit for G_3 is depicted in Fig. 5.

Considering the implementation of G_n . The purpose of G_n is to XOR all higher-order terms from the ANF forms onto the corresponding borrowed ancillary qubits. Our approach to implementing G_n is to start with G_2 and continuously use G_k to implement G_{k+1} , eventually achieving G_n . For implementing G_{k+1} based on G_k , we first use k Toffoli gates to implement all quadratic terms containing x_{k+1} (e.g., the three Toffoli gates in the dashed box in Fig. 6). Then, we add $2^k - k - 1$ Toffoli gates

Fig. 6. Quantum circuit of G_4 .

before and after the circuit to implement all other terms. Fig. 6 is an example circuit for implementing G_4 . The correctness of the circuit comes from the fact that all monomials generated by $k+1$ variables comprise are merely monomials generated by k variables along with these monomials multiplied by x_{k+1} .

To analyze circuit depth, we focus on the quantum gates on the left of the G_n gate. There will be at most $2^n - 1$ CNOT gates, acting on the target qubit $|t\rangle$ with distinct control qubits. Such a circuit can be implemented using a quantum parity gate with $O(n)$ circuit depth [47]. The parallel implementation of quantum gates between G_n and $G_n^\dagger$ follows a similar approach.

For circuit depth of G_n , note that when constructing the circuit for G_{k+1} , we add $2^k - 1$ Toffoli gates on the right side of the circuit for G_k . These Toffoli gates share a common control qubit x_{k+1} , while the other control qubits and target qubits are all distinct. Such a circuit can be implemented in parallel using quantum fan-out gates, as shown in Fig. 7. This shared-control Toffoli gate technique was proposed by Gokhale et al. [52]. Using this technique to implement $2^k - 1$ Toffoli gates, the circuit depth is primarily contributed by the four quantum fan-out gates $\mathfrak{F}_{2^{k-1}}$. According to Lemma 1, $2^k - 1$ Toffoli gates only require a circuit depth of $O(k)$ for implementation. Similarly, the same method can be used for the $2^k - k - 1$ Toffoli gates added on the left side of G_k . Therefore, G_n can be implemented with a quantum circuit of depth $O(n^2)$. This results in the whole circuit having a depth of $O(n^2)$. ■

Theorem 2: Any symmetric function $f(x)$ can be effectively implemented utilizing a quantum circuit that has a depth of $O(\log^2 n)$ and $\lceil \log(n+1) \rceil$ clean ancillary qubits.

Proof: First, we construct the ANF form of $g(|x|)$, where $g(|x|) = f(x)$. This step can be done by first constructing the truth table of $g(|x|)$, and then transforming the truth table into the ANF form mentioned in Lemma 3. The classical computation complexity can be bounded by $n^{O(1)}$ [53]. Define $m = \lceil \log(n+1) \rceil$. We use $|x|_1$ to denote the most significant bit of $|x|$, and $|x|_{2:m}$ to denote the remaining $m-1$ bits. The strategy is, we divide the terms of g into two parts based on whether they contain $|x|_1$, i.e., $g(|x|) = g_0(|x|_{2:m}) \oplus |x|_1 g_1(|x|_{2:m})$, where g_0 and g_1 are Boolean functions that depends on $m-1$ variables.

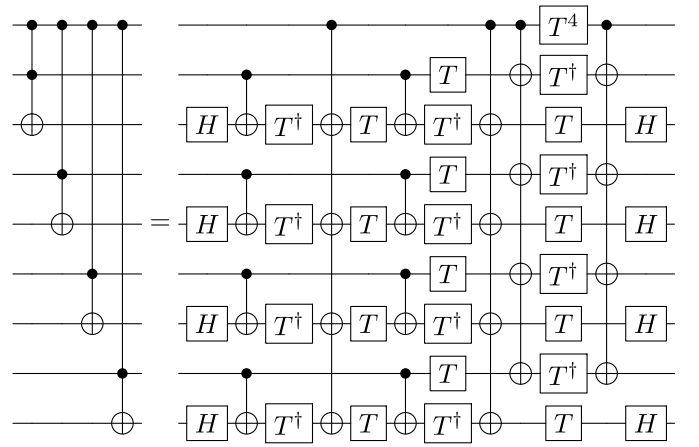


Fig. 7. Quantum circuit of four shared-control Toffoli gates.

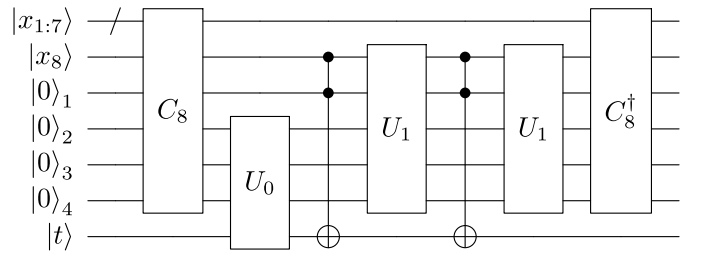
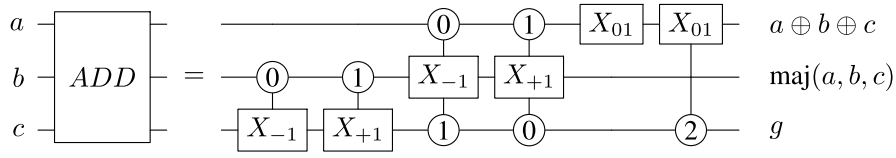


Fig. 8. Quantum circuit for an 8-bit symmetric function.

We construct the quantum circuit as follows. First, we create a circuit that prepares $|x|$ on m clean ancillary qubits, as described in Theorem 1. It is worth noting that by utilizing the n input qubits as borrowed ancillary qubits, we can implement g_0 and g_1 through Lemma 3. The quantum circuit continues as follows: compute the result of $g_0(|x|_{2:m})$ on $|t\rangle$ (the target qubit); add a Toffoli gate whose control qubits are $|x|_m$ and an input qubit $|a\rangle$ that not used in the previous step, target qubit is $|t\rangle$; compute the result of $g_1(|x|_{2:m})$ on $|a\rangle$; add a Toffoli gate same as the previous step; compute the result of $g_1(|x|_{2:m})$ on $|a\rangle$; recover the clean ancillary qubits. Our circuit first adds the result of $g_0(|x|_{2:m})$ to $|t\rangle$. Then, it uses a borrowed ancillary qubit $|a\rangle$ to add the result of $|x|_1 g_1(|x|_{2:m})$ to $|t\rangle$. Finally, we recover the state of $|a\rangle$ and m clean ancillary qubits.

To see the correctness of our circuit, the circuit of 8-bit symmetric function is depicted in Fig. 8. First, C_8 compute $|x| = \sum_{i=1}^8 x_i$ and store it in four clean ancillary qubits according to Theorem 1. Note that we have $f(x) = g(|x|) = g_0(|x|_{2:4}) \oplus |x|_1 g_1(|x|_{2:4})$. According to Lemma 3, we could construct U_0 to transform $|t\rangle$ to $|t \oplus g_0(|x|_{2:4})\rangle$, and construct U_1 to transform $|x_8\rangle$ to $|x_8 \oplus g_1(|x|_{2:4})\rangle$. Thus, the transformation of target qubit $|t\rangle$ is as follows:

$$\begin{aligned}
 |t\rangle &\rightarrow |t \oplus g_0(|x|_{2:4})\rangle, \text{ after } U_0 \\
 &\rightarrow |t \oplus g_0(|x|_{2:4}) \oplus x_8 |x|_1\rangle, \text{ after first Toffoli gate} \\
 &\rightarrow |t \oplus g_0(|x|_{2:4}) \oplus x_8 |x|_1 \oplus (x_8 \oplus g_1(|x|_{2:4})) |x|_1\rangle \\
 &= |t \oplus g_0(|x|_{2:4}) \oplus |x|_1 g_1(|x|_{2:4})\rangle \\
 &= |t \oplus f(x)\rangle, \text{ after second Toffoli gate.}
 \end{aligned}$$


 Fig. 9. Qutrit full adder. g represents garbage information.

The aim of computing $f(x)$ by two parts is to obtain enough borrowed ancillary qubits to use Lemma 3. The circuit depth is $O(\log^2 n)$ by Theorem 1 and Lemma 3. ■

IV. SYNTHESIS OF SYMMETRIC FUNCTIONS USING QUTRITS

In this section, we raise a method for the synthesis of symmetric functions with the help of higher energy levels. Concretely, given any symmetric function f , we can implement f in $O(\log^2 n)$ depth in qutrit systems using only 1 clean ancillary qutrit. Note that the input and output of f are still 0, 1-valued, and the extra energy level $|2\rangle$ is only for helping internal computing steps.

A. Computing Hamming Weight in Qutrit Systems

The key idea of our design is to utilize the extra energy level to compute the Hamming weight in place, that is, the Hamming weight is placed on the input qutrits instead of the extra clean ancillary qutrit. As a result, we can compute Hamming weight by $O(\log n)$ depth circuit in qutrit systems with one ancillary qutrit, which may be of independent interest.

We design a qutrit full adder with inputs $a, b, c \in \{0, 1\}$, represented by three qutrits. The circuit produces the outputs $a \oplus b \oplus c$ and $\text{majority}(a, b, c)$ as shown in Fig. 9. The circled number indicates the fire condition of the control qutrit. For example, the first gate in Fig. 9 indicates that when $|b\rangle = |0\rangle$, apply the X_{-1} gate to $|c\rangle$. The qutrit full adder can be decomposed into a constant number of elementary gates [10], and its correctness can be verified through straightforward computation.

We define sets $Q_0, Q_1, \dots, Q_t$, where $t = \lceil \log(n+1) \rceil - 1$. The n input qutrits are indexed by numbers in the set $\{1, 2, \dots, n\}$. An input qutrit may not belong to any set, or it may only belong to one set Q_i . The k th input qutrit belonging to Q_i is denoted by $k \in Q_i$. We want a qutrit in Q_i to contribute 2^i to the total sum if its state is $|1\rangle$. Thus, initially, $Q_0 = \{1, 2, \dots, n\}$ and $Q_i = \emptyset$ for $i \in \{1, 2, \dots, t\}$. Assuming the quantum state of the k th qutrit is $|y_k\rangle$, our desired equation is: $|x\rangle = \sum_{i=0}^t 2^i \sum_{k \in Q_i} [y_k = 1]$, where $x \in \{0, 1\}^n$ is the input of symmetric function. For indicator function $[y_k = 1]$, $[y_k = 1] = 1$ if and only if $y_k = 1$ otherwise $[y_k = 1] = 0$. Our aim is to apply quantum gates to input qutrits and make $|Q_i| = 1$ for every $i \in \{0, 1, \dots, t\}$. In this situation, the value of a qutrit belonging to Q_i is the i th bit of $|x\rangle$. This means that when the value is 1, it will contribute 2^i to $|x\rangle$.

It is worth noting that $a + b + c = a \oplus b \oplus c + 2 \cdot \text{majority}(a, b, c)$. This means that if we put three qutrits from Q_i into the qutrit full adder, it will contribute one qutrit to Q_i

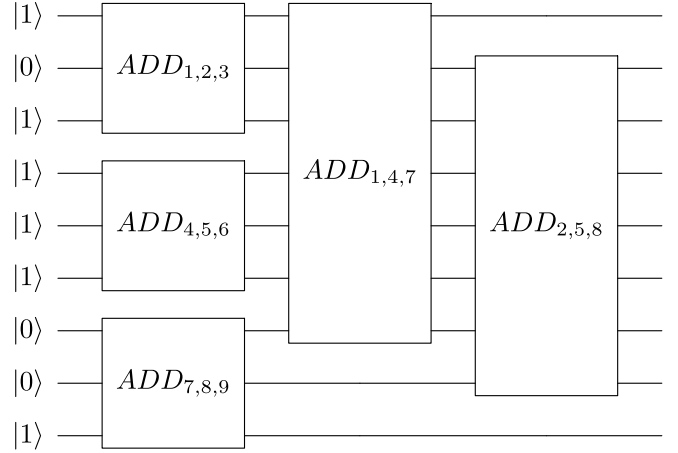


Fig. 10. Illustration of almost counting phase for nine input qutrits.

($a \oplus b \oplus c$) and one qutrit to Q_{i+1} ($\text{majority}(a, b, c)$). Therefore, we can repeatedly use the qutrit full adder to reduce the size of $|Q_i|$ if $|Q_i| \geq 3$. Algorithm 1 generates a quantum circuit to make $|Q_i| < 3$ for every set Q_i . The idea of Algorithm 1 is easy to understand. The key is to apply the qutrit full adder as many times as possible in parallel each time to reduce the value of $|Q_i|$ until it is no longer possible to apply the qutrit full adder and then stop. This is the first phase of counting, denoted as the almost counting phase. And Lemma 4 shows the property of the state of input qutrits after the almost counting phase.

An instance of the quantum circuit for almost counting phase with nine input qutrits in state $|101111001\rangle$ is illustrated in Fig. 10. In this circuit, $ADD_{i,j,k}$ means the qutrit full adder applied to the i, j, k th qutrits. Note that there are only two layers of qutrit full adders, as $ADD_{1,4,7}$ and $ADD_{2,5,8}$ can be applied in parallel. Table III shows how the state of qutrits and the sets Q_0, Q_1, Q_2 change when the circuit in Fig. 10 is executed.

Lemma 4: The circuit C generated by Algorithm 1 has the following properties: 1) with input $|x\rangle$ on computational basis, the output of C is pure state on computational basis denoted by $|y\rangle$, and $|x\rangle = \sum_{i=0}^t 2^i \sum_{k \in Q_i} [y_k = 1]$ and 2) there exists $0 \leq r \leq t$ such that $1 \leq |Q_0|, \dots, |Q_{r-1}| \leq 2$, and $|Q_r|, \dots, |Q_t| = 0$.

Proof: For the first condition, it is important to note that we only use qutrit full adder which is a reversible circuit. Therefore, if the input is a base state on computational basis then the output should be a base state on computational basis too. The equation in the first condition holds true at the beginning of the circuit. It can also be verified after applying any qutrit full adder. Regarding the second condition, we can

TABLE III

THIS TABLE SHOWS HOW THE STATE OF QUTRITS AND THE SETS Q_0, Q_1, Q_2 CHANGE WHEN THE CIRCUIT IN FIG. 10 IS EXECUTED. HERE, “g” MEANS GARBAGE INFORMATION THAT WE DON’T CARE ABOUT

Before the circuit is executed									
Qutrit number	1	2	3	4	5	6	7	8	9
State	1	0	1	1	1	1	0	0	1
$Q_0 = \{1, 2, 3, 4, 5, 6, 7, 8, 9\}, Q_1 = Q_2 = \emptyset$									
After apply one layer of qutrit full adders									
Qutrit number	1	2	3	4	5	6	7	8	9
State	0	1	g	1	1	g	1	0	g
$Q_0 = \{1, 4, 7\}, Q_1 = \{2, 5, 8\}, Q_2 = \emptyset$									
After the execution of the circuit									
Qutrit number	1	2	3	4	5	6	7	8	9
State	0	0	g	1	1	g	g	g	g
$Q_0 = \{1\}, Q_1 = \{2, 4\}, Q_2 = \{5\}$									

Algorithm 1 Generating Almost Counting Circuit in Qutrit Systems

```

1: Let  $C$  be an empty  $n$ -qutrit circuit whose qutrits are labeled by  $\{1, 2, \dots, n\}$ 
2:  $t \leftarrow \lceil \log(n+1) \rceil - 1, Q_0 \leftarrow \{1, 2, \dots, n\}, Q_1, \dots, Q_t \leftarrow \emptyset$ 
3: while  $\exists i, |Q_i| \geq 3$  do
4:    $M_0, \dots, M_{t-1} \leftarrow \emptyset$   $\triangleright$  Qubits in  $M_i$  will be carries
5:   for  $i = 0, 1, \dots, t-1$  do
6:     while  $|Q_i| \geq 3$  do
7:       Arbitrarily pick  $a, b, c \in Q_i$ 
8:       Add qutrit full adder on  $a, b, c$  to  $C$ 
9:        $M_i \leftarrow M_i \cup \{b\}$   $\triangleright$   $b$  stores majority
10:       $Q_i \leftarrow Q_i \setminus \{b, c\}$ 
11:      Let  $a$  will not be picked in this while loop
12:    end while
13:  end for
14:  for  $i = 0, 1, \dots, t-1$  do
15:     $Q_{i+1} \leftarrow Q_{i+1} \cup M_i$   $\triangleright$  Add carries to the next level
16:  end for
17: end while

```

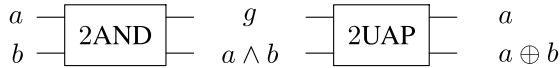


Fig. 11. 2-bit AND gate and “Un-And and Parity” (2UAP) gate. g represents garbage information.

make the following observations. First, the qutrit full adder cannot reduce $|Q_i|$ to zero if $|Q_i| > 0$. Second, the circuit from Algorithm 1 can reduce $|Q_i|$ if $|Q_i| > 2$. ■

The next step is the refining phase, where we aim to identify every Q_i such that $|Q_i| = 2$ and reduce its size to 1. In this phase, the quantum circuit resembles a ripple-carry adder [54]. If $|Q_i| = 2$, we generate a qutrit in Q_{i+1} from Q_i until we produce the most significant bit of $|x|$, which is stored in a clean ancillary qutrit. Then, we proceed to generate the remaining bits of $|x|$.

To generate a qutrit in Q_{i+1} from two qutrits in Q_i , we utilize a 2AND gate and a 2UAP (Un-And and Parity) gate. The functionality of these gates is illustrated in Fig. 11. The 2AND gate produces the logical AND of the two input bits, while the 2UAP gate generates the XOR of the input bits. To implement

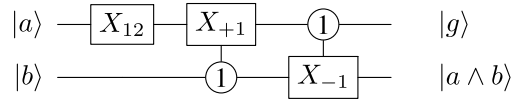


Fig. 12. Quantum circuit of 2AND.

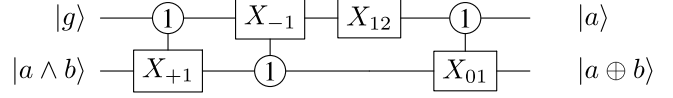


Fig. 13. Quantum circuit of 2UAP.

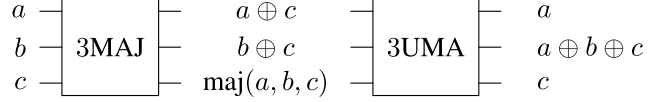


Fig. 14. 3-bit MAJORITY (3MAJ) and “Un-Majority and Add” (3UMA) gate in [54].

the 2AND gate, we consider the transformation $(a, b) \rightarrow (2a, b) \rightarrow (2a+b, b) \rightarrow (2a+b, b - [(2a+b) = 1])$. It can be observed that $(b - [(2a+b) = 1]) = a \wedge b$. Each step in this transformation can be implemented using elementary gates as shown in Fig. 12. The 2UAP gate can be implemented by reversing the 2AND gate and adding a $|1\rangle$ - X_{01} gate as shown in Fig. 13.

If $|Q_i| = 3$ after applying the 2AND gate to the qutrit in Q_{i-1} , we can utilize the 3MAJ (Majority) gate and 3UMA (Un-Majority and Add) gate [54], illustrated in Fig. 14, to achieve a similar functionality. 3MAJ and 3UMA have circuit implementations in qubit systems [54]. These circuits can be directly extended to qutrit systems and are therefore omitted. An example of the refining phase is illustrated in Fig. 15.

Theorem 3: There exists a quantum circuit of depth $O(\log n)$ in qutrit systems that utilizes one clean ancillary qutrit, such that, given input $|x, 0\rangle$, it computes the Hamming weight $|x|$, garbage, where the most significant bit of $|x|$ is stored in the ancillary qutrit.

Proof: The circuit consists of two phases: the almost counting phase and the refining phase, as described earlier. The correctness of the circuit follows directly from the discussion provided above. Note that in Algorithm 1, the gates generated in each iteration of the while loop (line 3) can be executed in parallel. For each layer of the qutrit full adder, there are no more than $2\lceil \log(n+1) \rceil$ qutrits that remain unchanged. Thus, each layer of the qutrit full adder reduces the $\sum_i |Q_i|$ roughly by $1/3$, resulting in the circuit depth generated by Algorithm 1 is $O(\log n)$. The second phase also has a depth of $O(\log n)$ since there are $O(\log n)$ qutrits in all the sets Q_i after the almost counting phase. This completes the proof. ■

In qutrit systems, there is a circuit with $O(\log n)$ depth that can compute the Hamming weight. It is natural to consider if there is also a circuit with $O(\log n)$ depth in qubit systems to compute the Hamming weight. Actually, based on Theorem 3, we can obtain Corollary 1. Note that Corollary 1 can also be seen as obtained by directly turning the implementation in [55] into a quantum circuit.

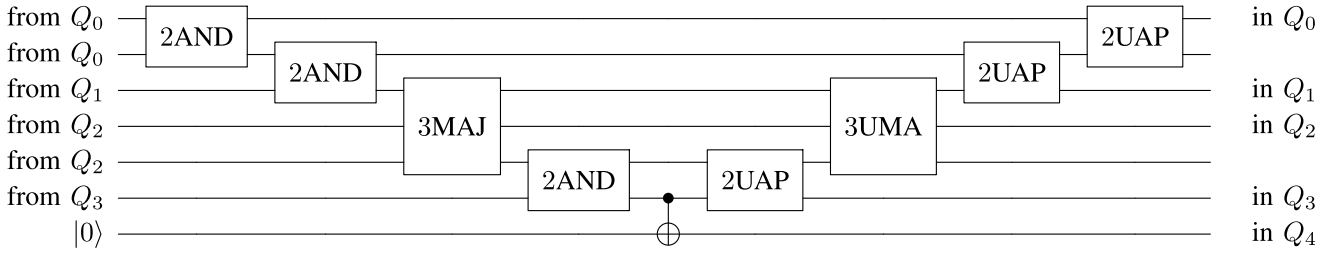


Fig. 15. Example for the refining phase of counting circuit in qutrit systems.

Corollary 1: In qubit systems, the Hamming weight of an n -bit string can be computed by a quantum circuit with depth $O(\log n)$, with $O(n)$ clean ancillary qubits.

Proof: We only need to transform the circuit in Theorem 3 into qubit systems. For the qutrit full adder in Fig. 9, we can design the qubit version as follows. First, use an ancillary qubit and three CNOTs to transform the state of the ancillary qubit to $|a \oplus b \oplus c\rangle$. Then, use the 3MAJ gate in Fig. 14 to obtain $\text{maj}(a, b, c)$. The 2AND gate in Fig. 12 can be implemented by an ancillary qubit and a Toffoli gate. For the 2UAP gate in Fig. 13, first use a Toffoli gate to recover $a \wedge b$. Then, use a CNOT gate to obtain $a \oplus b$. For the 3MAJ and 3UMA gates in Fig. 14, they do not need to change. Then, we use the same method shown in Theorem 3 to construct the circuit but use different gadgets as described above. The circuit depth is $O(\log n)$, and it uses $O(n)$ ancillary qubits. Note that when computing the Hamming weight $|x|$, we may destroy the input x . If we want to maintain the input x , we can initially use n ancillary qubits to store a copy of the n input qubits. The ancilla count is still $O(n)$. ■

In [56] and [57], they demonstrate that Hamming-weight measurement can be implemented by a circuit with $O(\log n)$ depth and $O(n)$ ancillary qubits by taking advantage of intermediate measurements and feedforward operations. Here, we show that the more general task of computing Hamming weight can be achieved with a circuit of $O(\log n)$ depth and $O(n)$ ancillary qubits, and it does not require intermediate measurements. Note that we only use the Clifford+T gate in Corollary 1. Thus, our result can be directly used in implementing equal-angle rotation through input weight calculation [44]. This is an important computing primitive in the fault-tolerant implementations of Hamiltonian dynamics simulations [45], [46].

B. Implementing Boolean Functions in Qutrit Systems

This section shows that any n -bit Boolean function can be implemented by a quantum circuit of depth $O(n^2)$ using $2^n - n - 1$ borrowed ancillary qutrit in qutrit systems. Our design is similar to the Boolean function implementation in Section III. The problem is that the borrowed ancillary qutrit we used is no longer suitable for operations over field $\mathbb{F}_2$. Luckily, we can still write any Boolean functions into polynomials in field $\mathbb{F}_3$.

Lemma 5: For any function $f : \{0, 1\}^n \rightarrow \{0, 1\}$, there are coefficients $c_S \in \{0, 1, 2\}$, $S \subseteq \{1, 2, \dots, n\}$ such that $f(x_1, \dots, x_n) = \sum_{S \subseteq \{1, 2, \dots, n\}} c_S \prod_{i \in S} x_i$, where the operations are over $\mathbb{F}_3$.

Proof: Denote $g_0(x_1, \dots, x_{n-1}) = f(x_1, \dots, x_{n-1}, 0)$ and $g_1(x_1, \dots, x_{n-1}) = f(x_1, \dots, x_{n-1}, 1)$. We have: $f(x_1, \dots, x_n) = (1 + 2x_n)g_0(x_1, \dots, x_{n-1}) + x_n g_1(x_1, \dots, x_{n-1})$.

By handling each input bit as x_n above, we can obtain the form $f(x_1, \dots, x_n) = \sum_{S \subseteq \{1, 2, \dots, n\}} c_S \prod_{i \in S} x_i$. ■

Lemma 6: Any Boolean function $f : \{0, 1\}^n \rightarrow \{0, 1\}$ can be implemented using a quantum circuit of depth $O(n^2)$ with $2^n - n - 1$ borrowed ancillary qutrit in qutrit systems.

Proof: The quantum circuit has the same structure as the circuit constructed in Lemma 3. According to Lemma 5, for the function $f(x_1, \dots, x_n) = \sum_{S \subseteq \{1, 2, \dots, n\}} c_S \prod_{i \in S} x_i$, we need to add every term in this function to the target qutrit to transform $|t\rangle$ to $|t + f(x)\rangle$.

For the linear term $c_S \prod_{i \in S} x_i$ where $|S| = 1$, we only need to add x_i or $2x_i$ to the target qutrit according to the coefficient. The transformation $|x, t\rangle \rightarrow |x, t + x\rangle$ can be implemented by two qutrit gates: $|1\rangle\text{-}X_{+1}$ and $|2\rangle\text{-}X_{-1}$. Applying this transformation twice results in $|x, t\rangle \rightarrow |x, t + 2x\rangle$. Thus, we can add all linear terms to the target qutrit $|t\rangle$.

For each higher-order term $T_S = c_S \prod_{i \in S} x_i$ where $|S| > 1$, we require a borrowed ancillary qutrit to add this term to the target qutrit $|t\rangle$. Assuming we can add term T_S to a borrowed ancillary qutrit $|a\rangle$, then T_S could be added to the target qutrit $|t\rangle$ as follows:

$$\begin{aligned} |a, t\rangle &\rightarrow |a, t + 2a\rangle \\ &\rightarrow |a + T_S, t + 2a\rangle \\ &\rightarrow |a + T_S, t + 2a + a + T_S\rangle \\ &= |a + T_S, t + T_S\rangle \\ &\rightarrow |a, t + T_S\rangle. \end{aligned}$$

Therefore, to add all higher-order terms to the target qutrit, we need $m = 2^n - n - 1$ borrowed ancillary qutrits $|a_1, a_2, \dots, a_m\rangle$. First, we add all the ancillary qutrits to the target qutrit twice

$$|a_1, a_2, \dots, a_m\rangle |t\rangle \rightarrow |a_1, a_2, \dots, a_m\rangle \left| t + \sum_{i=1}^m 2a_i \right\rangle.$$

Next, we add all the higher-order terms to the ancillary qutrit and then add all the ancillary qutrits to the target qutrit, transforming the target qutrit to $|t + \sum_{|S| > 1} T_S\rangle$. Finally, we recover all the ancillary qutrits, successfully adding all higher-order terms to the target qutrit. In practice, if $c_S = 0$, T_S does not need to be added to the target qutrit. If $c_S = 2$, $2 \prod_{i \in S} x_i$ is difficult to compute, so it is replaced by adding $\prod_{i \in S} x_i$ twice. Therefore, we assume $c_S = 1$ for all higher-order terms for convenience.

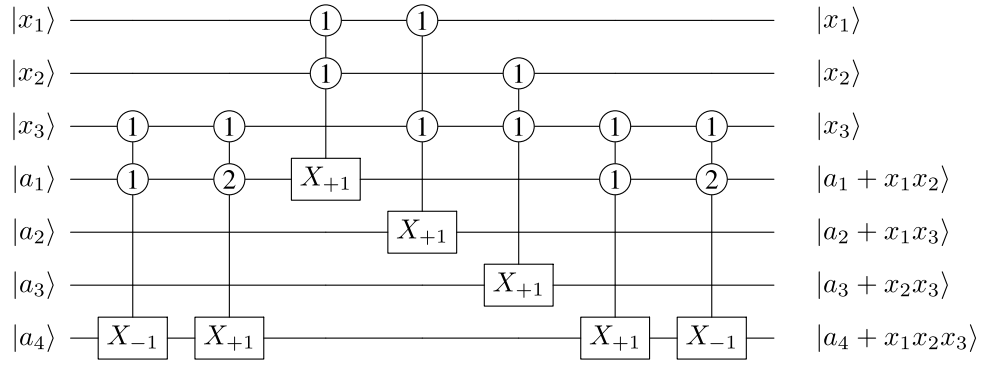
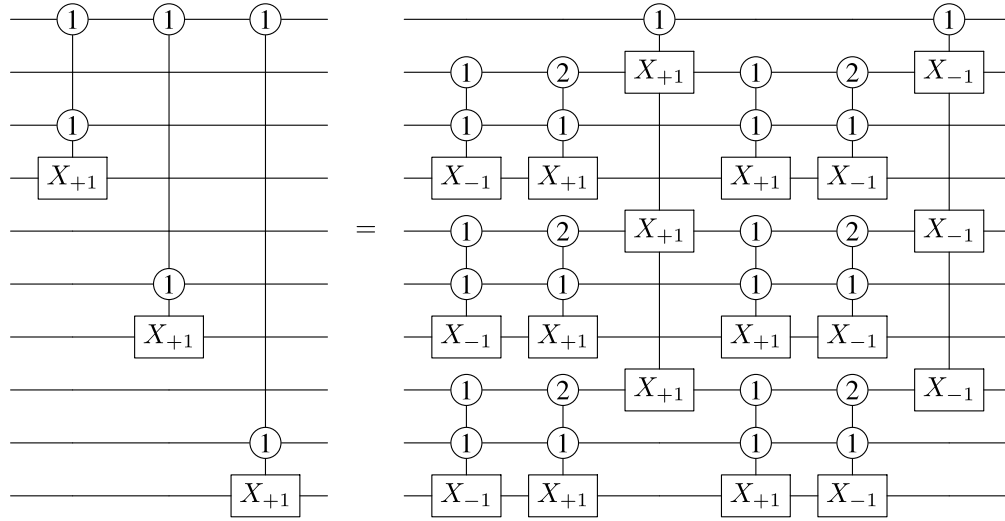
Fig. 16. Quantum circuit of $\mathbf{G}_3$ in qutrit.

Fig. 17. Quantum circuit of three shared-control qutrit gates.

For the constant term $c_\emptyset$, we apply the $X_{+c_\emptyset}$ gate to the target qutrit. Next, we introduce two key techniques to reduce the circuit depth.

We demonstrate the transformation

$$|a_1, \dots, a_m\rangle |t\rangle \rightarrow |a_1, \dots, a_m\rangle \left| t + \sum_i a_i \right\rangle$$

can be implemented by an $O(\log m) = O(n)$ depth circuit. Notably, we can add $|a_{2k}\rangle$ to $|a_{2k+1}\rangle$ for all $k = 0, 1, \dots$ using a constant layer of gates. This results in the need to add the value of $\lceil m/2 \rceil$ qutrits to the target qutrit only. Therefore, by employing a constant-depth circuit, we can halve the size of the problem, reducing the circuit depth to $O(\log m) = O(n)$.

We illustrate how to add all higher-order terms $T_S = \prod_{i \in S} x_i$, $|S| > 1$ to their corresponding ancillary qutrits using a $O(\log^2 m) = O(n^2)$ depth circuit. We set all coefficients c_S to 1 for convenience, as terms with $c_S = 2$ can be computed by adding them twice to the target qutrit. Let $\mathbf{G}_n$ denote the circuit that adds all n -bit higher-order terms to the ancillary qutrits, similar to the notation in Lemma 3. The quantum circuit $\mathbf{G}_3$ is illustrated in Fig. 16, which is similar to the circuit in Fig. 5. Let us consider how $\mathbf{G}_3$ adds $x_1x_2x_3$ to the ancillary qutrit

$$\begin{aligned} |x_3, a_1, a_4\rangle &\rightarrow |x_3, a_1, a_4 + 2a_1x_3\rangle \\ &\rightarrow |x_3, a_1 + x_1x_2, a_4 + 2a_1x_3\rangle \\ &\rightarrow |x_3, a_1 + x_1x_2, a_4 + 2a_1x_3 + (a_1 + x_1x_2)x_3\rangle \\ &= |x_3, a_1 + x_1x_2, a_4 + x_1x_2x_3\rangle. \end{aligned}$$

The above transformation illustrates how we can add $x_1x_2x_3$ to the ancillary qutrit if we can compute x_1x_2 . The key idea is that we can utilize the value of x_1x_2 that is added to $|a_1\rangle$ to compute the value of $x_1x_2x_3$. Similarly, we can add an arbitrary x_iP to an ancillary qutrit if we can compute P . Thus, we can extend the circuit of $\mathbf{G}_4$ in Fig. 6 to the qutrit systems to construct the circuit of $\mathbf{G}_4$ based on $\mathbf{G}_3$. Similarly, we can construct $\mathbf{G}_k$ based on $\mathbf{G}_{k-1}$ for arbitrary k . Note that the key technique of constructing $\mathbf{G}_k$ from $\mathbf{G}_{k-1}$ using an $O(k)$ depth circuit involves shared-control Toffoli gates, as shown in Fig. 7. Likewise, the key technique in the qutrit systems is the circuit of shared-control qutrit gates, an example of which is depicted in Fig. 17.

The key idea is to utilize borrowed ancillary qutrits to store the information of the shared-control qutrit. This necessitates a quantum fan-out gate in the qutrit systems to distribute the

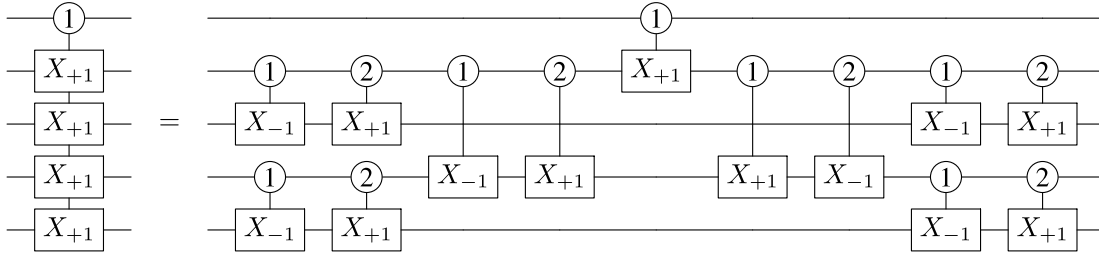


Fig. 18. Quantum circuit of quantum fan-out in qutrit systems.

value of the shared-control qutrit to all the ancillary qutrits. The m -qutrit fan-out gate can be implemented by an $O(\log m)$ depth circuit, with a proof similar to that of Lemma 1, which we omit here. An illustration of a 5-qutrit fan-out gate is provided in Fig. 18. In Fig. 18, if the control qutrit is in state $|1\rangle$, the target qutrits are applied X_{+1} gate. Reversing the circuit yields the circuit for applying X_{-1} to the target qutrit. In Fig. 17, we only depict the implementation of shared-control $|11\rangle$ - X_{+1} gates. The circuits for $|12\rangle$ - X_{+1} gate, $|11\rangle$ - X_{-1} gate, and $|12\rangle$ - X_{-1} gate are similar and omitted. Therefore, $\mathbf{G}_k$ can be constructed from $\mathbf{G}_{k-1}$ using an $O(k)$ depth circuit, similar to Lemma 3. Consequently, $\mathbf{G}_n$ can be implemented by an $O(n^2)$ depth circuit.

With all the techniques mentioned above, computing $f(\mathbf{x})$ involves four steps.

- 1) Adding all linear terms $c_S \prod_{i \in S} x_i$ where $|S| = 1$ to the target qutrit using an $O(n)$ depth circuit.
- 2) Adding higher-order terms $\prod_{i \in S} x_i$ where $|S| > 1$ and $c_S > 1$ to the target qutrit using an $O(n^2)$ depth circuit with $2^n - n - 1$ borrowed ancillary qutrit.
- 3) Adding higher-order terms $\prod_{i \in S} x_i$ where $|S| > 1$ and $c_S = 2$ to the target qutrit using an $O(n^2)$ depth circuit with $2^n - n - 1$ borrowed ancillary qutrit.
- 4) Adding the constant term $c_{\emptyset}$ to the target qutrit using the $X_{+c_{\emptyset}}$ gate.

As a result, the depth of the entire circuit is $O(n^2)$. To construct the circuit of $\mathbf{G}_n$, $2^n - n - 1$ borrowed ancillary qutrits are required. ■

With Lemma 6, we can now state Theorem 4.

Theorem 4: Any symmetric function $f(\mathbf{x})$ can be implemented using a quantum circuit of depth $O(\log^2 n)$ with one clean ancillary qutrit in qutrit systems.

Proof: Similar to the proof of Theorem 2, we first compute the Hamming weight of the input $|\mathbf{x}\rangle$ using Theorem 3. Then, we add the output $f(\mathbf{x})$ to the target qutrit in two parts, depending on whether the most significant bit of $|\mathbf{x}\rangle$ is 0 or 1. This division of the computation of $f(\mathbf{x})$ into two parts is done to ensure we have enough borrowed ancillary qutrits, as required by Lemma 6.

Specifically, let us assume $f(\mathbf{x}) = g(|\mathbf{x}\rangle) = g_0 + |\mathbf{x}|_1 g_1$, where $|\mathbf{x}|_1$ represents the most significant bit of $|\mathbf{x}\rangle$, and g_0 and g_1 are functions of $|\mathbf{x}|_{2:\lceil \log(n+1) \rceil}$. After computing the Hamming weight $|\mathbf{x}|$ using Theorem 3, the transformation for computing $f(\mathbf{x})$ to the target qutrit $|t\rangle$ proceeds as follows, where $|a\rangle$ is an input qutrit serving as borrowed ancillary qutrit:

$$||\mathbf{x}|_1, a, t\rangle$$

$$\rightarrow ||\mathbf{x}|_1, a, t + g_0\rangle, \text{Lemma 6}$$

$$\rightarrow ||\mathbf{x}|_1, a, t + g_0 + 2a|\mathbf{x}|_1\rangle$$

$$\rightarrow ||\mathbf{x}|_1, a + g_1, t + g_0 + 2a|\mathbf{x}|_1\rangle, \text{Lemma 6}$$

$$\rightarrow ||\mathbf{x}|_1, a + g_1, t + g_0 + 2a|\mathbf{x}|_1 + (a + g_1)|\mathbf{x}|_1\rangle$$

$$= ||\mathbf{x}|_1, a + g_1, t + g_0 + g_1|\mathbf{x}|_1\rangle$$

$$= ||\mathbf{x}|_1, a + g_1, t + f(\mathbf{x})\rangle$$

$$\rightarrow ||\mathbf{x}|_1, a, t + f(\mathbf{x})\rangle, \text{Recover } |a\rangle.$$

Constructing the quantum circuit based on the transformation above is straightforward and similar to Theorem 2, thus it is omitted. As a result, the circuit depth is $O(\log^2 n)$ according to Theorem 3 and Lemma 6. One clean ancillary qutrit is required in Theorem 3. ■

V. ANALYSIS AND EVALUATION

In this section, we provide an analysis of the depth of the quantum circuit that implements the Hamming weight and symmetric functions in qubit systems. We also evaluate the depth of the circuit that computes the majority function. It is important to note that all circuits are composed of CNOT and single-qubit gates. We omit a detailed analysis and restate the following lemma and theorem.

Lemma 7 (Lemma 1 Restate): The quantum fan-out gate $\mathfrak{F}_n$ can be implemented using a quantum circuit of depth $2\lceil \log n \rceil + 1$ without ancillary qubit.

Theorem 5 (Theorem 1 Restate): The Hamming weight of an n -bit string can be computed by a quantum circuit C_n with depth $4\lceil \log n \rceil \lceil \log(n+1) \rceil + 8\lceil \log(n+1) \rceil - 2$, without using ancillary qubits.

Proof: Note that the depth of the k -qubit quantum Fourier transform (QFT) is $5k - 4$, achieved by decomposing each controlled rotation gate into two CNOT gates and two single-qubit rotation gates. ■

Lemma 8 (Lemma 3 Restate): Any Boolean function $f : \{0, 1\}^n \rightarrow \{0, 1\}$ can be implemented using a quantum circuit of depth no more than $16n^2 + 36n - 154$ with $2^n - n - 1$ borrowed ancillary qubits.

Proof: Upon analyzing the circuit construction detailed in Lemma 3, we find that the depth of k shared-control Toffoli gates is $8\lceil \log k \rceil + 12$, and the depth of G_n is $8n^2 + 16n - 76$ for $n \geq 5$. Utilizing these analyses, we determine the depth of the circuit that computes the Boolean function f . ■

TABLE IV

THIS TABLE DISPLAYS THE CIRCUIT DEPTH REQUIRED TO COMPUTE THE HAMMING WEIGHT, MAJORITY FUNCTION, AND ARBITRARY SYMMETRIC FUNCTIONS WITH VARYING NUMBERS OF INPUT QUBITS. ADDITIONALLY, IT INDICATES THE NUMBER OF ANCILLARY QUBITS NEEDED TO COMPUTE SYMMETRIC FUNCTIONS

Number of input qubits	31	63	127	255	511	1023	2047	4095
Depth of circuit computing Hamming weight	138	190	250	318	394	478	570	670
Depth of circuit computing majority function	277	381	501	637	789	957	1141	1341
Depth of circuit computing symmetric functions	≤ 1038	≤ 1682	≤ 2438	≤ 3306	≤ 4286	≤ 5378	≤ 6582	≤ 7898
Ancilla count of circuit computing symmetric functions	5	6	7	8	9	10	11	12

TABLE V

THIS TABLE SHOWS THE NUMBER OF LAYERS OF QUTRIT FULL ADDERS REQUIRED TO COMPUTE THE HAMMING WEIGHT AND MAJORITY FUNCTION WITH VARYING NUMBERS OF INPUT QUBITS WITHOUT ANCILLARY QUTRIT

Number of input qutrits	31	63	127	255	511	1023	2047	4095
Number of layers of qutrit full-adders to compute Hamming weight	7	9	10	12	14	16	18	20
Number of layers of qutrit full-adders to compute majority function	14	18	20	24	28	32	36	40

TABLE VI

THIS TABLE PRESENTS THE SPECIFIC COSTS OF THE QUANTUM CIRCUIT, WHICH IS CONSTRUCTED USING OUR DISTINCT METHODS FOR COMPUTING THE HAMMING WEIGHT. THREE DIFFERENT METHODS ARE SHOWN: THEOREM 1, COROLLARY 1, AND THEOREM 3. FOR EACH METHOD, THE CIRCUIT DEPTH, CIRCUIT SIZE, AND THE COUNT OF ANCILLA QUBITS ARE INDICATED

Number of input qubits/qutrits	5	10	25	50	75	100	150	200
Theorem 1: Depth	58	94	138	190	250	250	318	318
Theorem 1: Size	93	234	680	1581	2737	3612	6148	8148
Theorem 1: Ancilla count	3	4	5	6	7	7	8	8
Corollary 1: Depth	39	88	136	217	207	272	298	311
Corollary 1: Size	70	194	474	998	1464	2008	3002	4018
Corollary 1: Ancilla count	3	7	21	45	70	95	143	195
Theorem 3: Depth	31	52	124	193	203	248	262	279
Theorem 3: Size	54	146	482	1054	1640	2236	3402	4618
Theorem 3: Ancilla count	1	1	1	1	1	1	1	1

Theorem 6 (Theorem 2 Restate): Any symmetric function $f(x)$ can be effectively implemented utilizing a quantum circuit whose depth is no more than $48(\lceil \log(n+1) \rceil)^2 + 8\lceil \log(n+1) \rceil \lceil \log n \rceil + 28\lceil \log(n+1) \rceil - 502$, with $\lceil \log(n+1) \rceil$ clean ancillary qubits.

Here, we consider the depth of the circuit constructed by our method for various numbers of input qubits, specifically when $n \in \{31, 63, 127, 255, 511, 1023, 2047, 4095\}$. The depth of the circuit computing the Hamming weight can be directly obtained using Theorem 5.

The majority function is a Boolean function that outputs 1 if and only if at least half of the input bits are 1. In our setup, where the number of input qubits has the form $2^k - 1$ for some k , the output of the majority function is the most significant bit of the Hamming weight of the input. Therefore, we can compute the majority function by computing the Hamming weight and copying the most significant bit to the output qubit.

For arbitrary symmetric functions, Theorem 6 provides an upper bound on the circuit depth, as the depth varies depending on the specific symmetric function. The evaluation results are presented in Table IV. The results demonstrate that we can compute the Hamming weight and majority function with extremely low depth, even when the number of input qubits is large.

In qutrit systems, it is worth noting that when the number of input qubits has the form $2^k - 1$, the Hamming weight can be computed solely by Algorithm 1 without ancillary qutrit. We evaluate the number of layers of the full adder (depicted in

Fig. 9) needed to compute the Hamming weight and majority function with different numbers of input qubits. The results are shown in Table V.

The above concrete analysis and evaluation have demonstrated that our circuit construction method has great performance as shown by theoretical proof. To show the practicality of our methods, we build a benchmark to display the cost of our three different methods for constructing the circuit to compute Hamming weight. We display the circuit depth, circuit size, and ancilla count in Table VI. By this benchmark, our methods prove practical when the problem scale is small. The three methods are explicitly explained as follows.

The first method is Theorem 1, the basic gate set we use is arbitrary single qubit gates and CNOT gate. The second method is Corollary 1, and the basic gate set we use is Clifford+ T gates. The first two methods are used in qubit systems. The third method is Theorem 3, which is used in qutrit systems. For convenience, let the basic gate set contain arbitrary single qutrit gates and one-controlled single qutrit gates.

VI. CONCLUSION

We introduce a novel method for implementing arbitrary symmetric function using a quantum circuit with only $O(\log^2 n)$ depth and $\lceil \log(n+1) \rceil$ ancillary qubits. The key technique is an $O(\log^2 n)$ depth circuit design to compute

Hamming weight without ancillary qubit. Remarkably, in qutrit systems, the number of ancilla can be reduced to just one. Our results demonstrate that symmetric functions can be implemented by quantum circuits with both low depth and low ancilla count. Additionally, our results partially address the question of space–time tradeoffs in quantum circuits raised by Maslov et al. [5].

For future research, we are curious about whether the circuit depth can be further reduced to $O(\log n)$ to achieve asymptotic optimality and maintain a low cost of ancillary qubits. We are also interested in exploring the implementation of other classes of Boolean functions using poly-logarithmic or sublinear depth circuits. Moreover, the fault-tolerant circuit implementation of symmetric functions is intriguing.

ACKNOWLEDGMENT

The authors would like to express their sincere gratitude to the reviewers. Their careful and constructive comments have been of great assistance to them, and have significantly improved the quality of this article. They are also grateful to David Raveh for his concern regarding their results and for bringing [57] to their attention.

REFERENCES

- [1] P. W. Shor, “Algorithms for quantum computation: Discrete logarithms and factoring,” in *Proc. 35th Annu. Symp. Found. Comput. Sci.*, 1994, pp. 124–134.
- [2] M. Amy, D. Maslov, M. Mosca, and M. Roetteler, “A meet-in-the-middle algorithm for fast synthesis of depth-optimal quantum circuits,” *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 32, no. 6, pp. 818–830, Jun. 2013.
- [3] X. Sun, G. Tian, S. Yang, P. Yuan, and S. Zhang, “Asymptotically optimal circuit depth for quantum state preparation and general unitary synthesis,” *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 42, no. 10, pp. 3301–3314, Oct. 2023.
- [4] R. Vale, T. M. D. Azevedo, I. C. S. Araújo, I. F. Araújo, and A. J. Da Silva, “Circuit decomposition of multicontrolled special unitary single-qubit gates,” *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 43, no. 3, pp. 802–811, Mar. 2024.
- [5] D. Maslov, J.-S. Kim, S. Bravyi, T. J. Yoder, and S. Sheldon, “Quantum advantage for computations with limited space,” *Nat. Phys.*, vol. 17, no. 8, pp. 894–897, 2021.
- [6] J. Jiang, X. Sun, S.-H. Teng, B. Wu, K. Wu, and J. Zhang, “Optimal space-depth trade-off of CNOT circuits in quantum logic synthesis,” in *Proc. 14th Annu. ACM-SIAM Symp. Discrete Algorithms*, 2020, pp. 213–229.
- [7] Y. Takahashi and S. Tani, “Power of uninitialized qubits in shallow quantum circuits,” *Theor. Comput. Sci.*, vol. 851, pp. 129–153, Jan. 2021.
- [8] A. Chattopadhyay and A. Baksı, “Low-quantum cost circuit constructions for adder and symmetric boolean functions,” in *Proc. IEEE Int. Symp. Circuits Syst. (ISCAS)*, 2016, pp. 2294–2297.
- [9] P. Gokhale, J. M. Baker, C. Duckering, N. C. Brown, K. R. Brown, and F. T. Chong, “Asymptotic improvements to quantum circuits via qutrits,” in *Proc. 46th Int. Symp. Comput. Archit. (ISCA)*, 2019, pp. 554–566.
- [10] W. Zi, Q. Li, and X. Sun, “Optimal synthesis of multi-controlled qutrit gates,” in *Proc. 60th ACM/IEEE Design Autom. Conf. (DAC)*, 2023, pp. 1–6.
- [11] J. Nie, Q. Zhu, M. Li, and X. Sun, “Quantum circuit design for integer multiplication based on Schönhage-Strassen algorithm,” *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 42, no. 12, pp. 4791–4802, Dec. 2023.
- [12] X. He, X. Sun, and J. Zhang, “Quantum search with prior knowledge,” *Sci. China Inf. Sci.*, vol. 67, no. 9, 2024, Art. no. 192503.
- [13] A. Canteaut and M. Videau, “Symmetric boolean functions,” *IEEE Trans. Inf. Theory*, vol. 51, no. 8, pp. 2791–2811, Aug. 2005.
- [14] G. Yang, W. N. Hung, X. Song, and M. Perkowski, “Majority-based reversible logic gates,” *Theor. Comput. Sci.*, vol. 334, nos. 1–3, pp. 259–274, 2005.
- [15] L. Biswal, B. Mondal, A. Chakraborty, and H. Rahaman, “Efficient quantum implementation of majority-based full adder circuit using clifford+T-group,” in *Proc. Artif. Intell. Driven Circuits Syst., Select (ISED)*, 2022, pp. 53–63.
- [16] M. A. Nashiry and J. E. Rice, “A reversible majority voter circuit and applications,” in *Proc. IEEE Pac. Rim Conf. Commun., Comput. Signal Process. (PACRIM)*, 2017, pp. 1–6.
- [17] A. Deb, D. K. Das, H. Rahaman, R. Wille, R. Drechsler, and B. B. Bhattacharya, “Reversible synthesis of symmetric functions with a simple regular structure and easy testability,” *ACM J. Emerg. Technol. Comput. Syst.*, vol. 12, no. 4, pp. 1–29, Jun. 2016. [Online]. Available: <https://doi.org/10.1145/2894757>
- [18] A. Deb, D. K. Das, H. Rahaman, B. B. Bhattacharya, R. Wille, and R. Drechsler, “Reversible circuit synthesis of symmetric functions using a simple regular structure,” in *Proc. 5th Int. Conf. Revers. Comput. (RC)*, 2013, pp. 182–195.
- [19] M. Perkowski et al., “Regular realization of symmetric functions using reversible logic,” in *Proc. Euromicro Symp. Digit. Syst. Design*, 2001, pp. 245–252.
- [20] M. Perkowski and L. Jozwiak, “English Regularity and symmetry as a base for efficient realization of reversible logic circuits,” in *Proc. 10th Int. Workshop Logic Synth. (IWLS)*, 2001, pp. 90–95.
- [21] D. A. Maslov, “Dynamic programming algorithms as quantum circuits: Symmetric function realization,” in *Proc. 2nd Quantum Inf. Comput.*, 2004, pp. 386–393.
- [22] D. Maslov, “Efficient reversible and quantum implementations of symmetric boolean functions,” *IEE Proc.-Circuits, Devices Syst.*, vol. 153, no. 5, pp. 467–472, 2006.
- [23] M. Chrzanowska-Jeske, Y. Xu, and M. Perkowski, “Logic synthesis for a regular layout,” *VLSI Design*, vol. 10, no. 1, pp. 35–55, 1999.
- [24] D. T. Lee and S. J. Hong, “An algorithm for transformation of an arbitrary switching function to a completely symmetric function,” *IEEE Trans. Comput.*, vol. C-25, no. 11, pp. 1117–1123, Nov. 1976.
- [25] M. Khan, J. P. L. Faye, U. C. Mendes, and A. Miranskyy, “EP-PQM: Efficient parametric probabilistic quantum memory with fewer qubits and gates,” *IEEE Trans. Quantum Eng.*, vol. 3, pp. 1–15, Apr. 2022.
- [26] K. Kathuria, A. Ratan, M. McConnell, and S. Bekiranov, “Implementation of a hamming distance-like genomic quantum classifier using inner products on ibmqx2 and ibmq_16_melbourne,” *Quantum Mach. Intell.*, vol. 2, no. 1, p. 7, 2020.
- [27] G. Li, X. Zhao, and X. Wang, “Quantum self-attention neural networks for text classification,” *Sci. China Inf. Sci.*, vol. 67, no. 4, 2024, Art. no. 142501.
- [28] Y. Ruan, X. Xue, H. Liu, J. Tan, and X. Li, “Quantum algorithm for K-nearest neighbors classification based on the metric of hamming distance,” *Int. J. Theor. Phys.*, vol. 56, pp. 3496–3507, Aug. 2017.
- [29] J. Li, S. Lin, K. Yu, and G. Guo, “Quantum K-nearest neighbor classification algorithm based on hamming distance,” *Quantum Inf. Process.*, vol. 21, no. 1, p. 18, 2022.
- [30] A. Berti, A. Bernasconi, G. M. Del Corso, and R. Guidotti, “Effect of different encodings and distance functions on quantum instance-based classifiers,” in *Proc. 26th Pac.-Asia Conf. Knowl. Discov. Data Min.*, 2022, pp. 96–108.
- [31] J. Li et al., “Quantum k-fold cross-validation for nearest neighbor classification algorithm,” *Physica A, Stat. Mech. Appl.*, vol. 611, Feb. 2023, Art. no. 128435.
- [32] Z. Tian and S. Baskiyar, “Fake news detection: An application of quantum k-nearest neighbors,” in *Proc. IEEE Symp. Ser. Comput. Intell. (SSCI)*, 2021, pp. 1–6.
- [33] F. Orts, G. Ortega, E. F. Combarro, I. F. Rúa, and E. M. Garzón, “Quantum circuits for computing hamming distance requiring fewer T gates,” *J. Supercomput.*, vol. 80, pp. 12527–12542, Feb. 2024.
- [34] M. Ringbauer et al., “A universal qutrit quantum processor with trapped ions,” *Nat. Phys.*, vol. 18, no. 9, pp. 1053–1057, 2022.
- [35] Y. Chi et al., “A programmable qutrit-based quantum processor,” *Nat. Commun.*, vol. 13, no. 1, p. 1166, 2022.
- [36] L. E. Fischer, A. Chiesa, F. Tacchino, D. J. Egger, S. Carretta, and I. Tavernelli, “Universal qutrit gate synthesis for transmons,” *PRX Quantum*, vol. 4, no. 3, 2023, Art. no. 30327.
- [37] P. Liu et al., “Performing SU(d) operations and rudimentary algorithms in a superconducting transmon qutrit for d=3 and d=4,” *Phys. Rev. X*, vol. 13, no. 2, 2023, Art. no. 21028.

- [38] K. Luo et al., "Experimental realization of two qutrits gate with tunable coupling in superconducting circuits," *Phys. Rev. Lett.*, vol. 130, no. 3, 2023, Art. no. 30603.
- [39] S. Omanakuttan, A. Mitra, E. J. Meier, M. J. Martin, and I. H. Deutsch, "Qudit entanglers using quantum optimal control," *PRX Quantum*, vol. 4, no. 4, 2023, Art. no. 40333.
- [40] P. Hrmo et al., "Native qudit entanglement in a trapped ion quantum processor," *Nat. Commun.*, vol. 14, no. 1, p. 2242, 2023.
- [41] Y. Guo et al., "Single-shot readout of a solid-state electron spin qutrit," *Phys. Rev. Lett.*, vol. 132, no. 6, 2024, Art. no. 60601.
- [42] S. S. Bullock, D. P. O'Leary, and G. K. Brennen, "Asymptotically optimal quantum circuits for d-level systems," *Phys. Rev. Lett.*, vol. 94, no. 23, 2005, Art. no. 230502.
- [43] J. Chu et al., "Scalable algorithm simplification using quantum AND logic," *Nat. Phys.*, vol. 19, no. 1, pp. 126–131, 2023.
- [44] C. Gidney, "Halving the cost of quantum addition," *Quantum*, vol. 2, p. 74, Jun. 2018.
- [45] Y. Nam and D. Maslov, "Low-cost quantum circuits for classically intractable instances of the hamiltonian dynamics simulation problem," *Npj Quantum Inf.*, vol. 5, no. 1, p. 44, 2019.
- [46] I. D. Kivlichan et al., "Improved fault-tolerant quantum simulation of condensed-phase correlated electrons via trotterization," *Quantum*, vol. 4, p. 296, Jul. 2020.
- [47] M. Fang, S. Fenner, F. Green, S. Homer, and Y. Zhang, "Quantum lower bounds for fanout," *Quantum Inf. Comput.*, vol. 6, no. 1, pp. 46–57, 2006.
- [48] M. A. Nielsen and I. L. Chuang, *Quantum Computation and Quantum Information*. Cambridge, U.K.: Cambridge Univ. Press, 2010.
- [49] T. G. Draper, "Addition on a quantum computer," 2000, *arXiv:quant-ph/0008033*.
- [50] Y. Wang and B. M. Terhal, "Preparing Dicke states in a spin ensemble using phase estimation," *Phys. Rev. A*, vol. 104, no. 3, 2021, Art. no. 32407.
- [51] A. Chattopadhyay and A. Baksi, "Reversible logic circuit complexity analysis via functional decomposition," 2016, *arXiv:1602.00101*.
- [52] P. Gokhale et al., "Quantum fan-out: Circuit optimizations and technology modeling," in *Proc. IEEE Int. Conf. Quantum Comput. Eng. (QCE)*, 2021, pp. 276–290.
- [53] T. Sasao, "EXMIN2: A simplification algorithm for exclusive-OR-sum-of-products expressions for multiple-valued-input two-valued-output functions," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 12, no. 5, pp. 621–632, May 1993.
- [54] S. A. Cuccaro, T. G. Draper, S. A. Kutin, and D. P. Moulton, "A new quantum ripple-carry addition circuit," 2004, *arXiv:quant-ph/0410184*.
- [55] I. Wegener, *The Complexity of Boolean Functions*. Hoboken, NJ, USA: Wiley, 1987.
- [56] S. Rethinasamy, M. L. LaBorde, and M. M. Wilde, "Logarithmic-depth quantum circuits for hamming weight projections," *Phys. Rev. A*, vol. 110, no. 5, 2024, Art. no. 52401.
- [57] L. Piroli, G. Styliaris, and J. I. Cirac, "Approximating many-body quantum states with quantum circuits and measurements," *Phys. Rev. Lett.*, vol. 133, no. 23, 2024, Art. no. 230401.